AgenticSec: AI Code Reviewer

An LLM-driven DevSecOps Pipeline for Automated Vulnerability Remediation

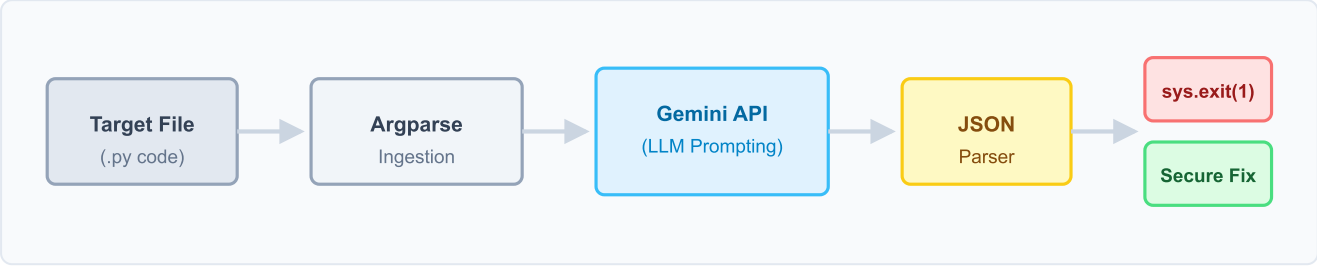
1. Executive Summary

In modern software engineering, security is often treated as a final barrier before deployment, leading to bottlenecks, delayed releases, and friction between security and development teams. The objective of this project is to shift security to the absolute "left" of the software development lifecycle (SDLC) by implementing a localized, AI-powered pre-commit scanner.

By leveraging the Gemini API and agentic prompting techniques, this tool dynamically ingests source code, performs contextual static analysis for OWASP Top 10 vulnerabilities, and outputs actionable, drop-in remediation code. This eliminates the research burden on developers, significantly accelerating secure feature delivery.

2. System Architecture

The architecture is designed as a lightweight, dependency-minimal Python application. It operates entirely locally or inside a CI/CD runner, ensuring high portability. A strict JSON schema enforcement guarantees that the AI acts deterministically, allowing mathematical evaluation of the risk severity rather than relying on qualitative text analysis.



3. Core Features

Dynamic Ingestion

Replaces hardcoded logic with Python's `argparse` and `sys` libraries, allowing the tool to accept dynamic file paths at runtime. This enables integration into pre-existing developer workflows and automated terminal hooks.

Build-Breaker Gate

Acts as an automated CI/CD guardrail. By parsing the AI's JSON output for the `severity` key, the script triggers a `sys.exit(1)` kill-signal if High or Critical flaws are found, physically preventing vulnerable code from merging.

Auto-Remediation

Moves beyond traditional static scanning (SAST) by generating secure, context-aware patches. The AI is prompted to return a `remediation_code` key containing the exact syntax required to resolve the identified security vulnerability.

4. Prerequisites & Setup

The tool is designed for zero-friction adoption. It requires Python 3.8+ and an active API key from Google AI Studio. No database or complex virtualization is required.

ENVIRONMENT SETUP (TERMINAL)

```
pip install google-generativeai argparse export GEMINI_API_KEY="your_api_key_here"
```

EXECUTION EXAMPLE

```
# Scan a local python file python agentic_reviewer.py ./src/database_queries.py # Expected Output on Failure [Critical] SQL Injection on line 14: Unsanitized input in f-string query. --- SUGGESTED FIX --- cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,)) ---------- ❌ CRITICAL VULNERABILITY FOUND. FAILING THE BUILD.
```

5. Future Enhancements & Roadmap

While the current architecture successfully functions as a local, single-file DevSecOps tool, it can be extended to operate at an enterprise scale. The modular nature of the JSON-parsing logic allows for seamless integration into broader automation ecosystems.



1. **GitHub Actions Integration (CI/CD Pipeline):** Porting the script to execute automatically inside a GitHub Actions runner upon Pull Request creation. The script will use the GitHub API (`PyGithub`) to post the AI's `remediation_code` directly as inline PR comments.
2. **Repository-Wide Context:** Currently, the tool analyzes single files in isolation. Future iterations will utilize RAG (Retrieval-Augmented Generation) to give the LLM context of the entire codebase, reducing false positives related to imported custom sanitization libraries.

6. Conclusion

The *AgenticSec Code Reviewer* demonstrates a practical application of Generative AI in cybersecurity operations. By combining traditional pipeline gating mechanisms (exit codes) with advanced NLP reasoning and strict data formatting (JSON schemas), this project bridges the gap between identifying risk and actively mitigating it.

> **Project Impact Statement:**
>
> This architecture transitions the security paradigm from "detect and report" to "detect and resolve," minimizing developer downtime and showcasing advanced proficiency in automation, AI integration, and core AppSec methodologies.